IPL-INTERNSHIP PROJECT

Althaf K A

Introduction:

The Indian Premier League (IPL) is one of the most popular and commercially successful cricket tournaments in the world. Established in 2008 by the Board of Control for Cricket in India (BCCI), it revolutionized cricket by introducing a franchise-based Twenty20 format. The IPL brings together international and domestic players, creating a platform that blends sports, entertainment, and business. Over the years, it has grown into a global spectacle, attracting millions of viewers, generating significant revenue, and influencing the way cricket is played and consumed. Its impact extends beyond the field, shaping sponsorship models, broadcasting rights, and fan engagement strategies.

The IPL Prediction System is a full-stack application designed to deliver accurate, scalable, and production-ready predictions for Indian Premier League (IPL) player runs. Built with a robust backend powered by FastAPI, Streamlit, MLflow, and containerized using Docker Compose, the project integrates modern machine learning workflows with seamless deployment pipelines via GitHub Actions.

At its core, the system leverages polynomial regression models trained on key batting and match features (such as Inns, NO, Avg, BF, 4s, and 6s) to forecast player and team performance. The architecture emphasizes maintainability, developer convenience, and observability, with integrated monitoring through Prometheus and Grafana to ensure reliability in production environments.

Problem statement:

Despite its immense popularity, the IPL faces several challenges that make it an interesting subject for study and analysis:

- Player Performance Analysis: With hundreds of matches played, evaluating player consistency and predicting future performance is complex.
- Team Strategy Optimization: Franchise teams must balance star players, emerging talent, and budget constraints while aiming for competitive success.
- Fan Engagement & Viewership Trends: Understanding audience preferences, match attendance, and digital engagement is crucial for sustaining growth.
- Financial & Commercial Aspects: Managing sponsorships, broadcasting rights, and franchise valuations requires data-driven insights.
- Scheduling & Logistics: Coordinating matches across multiple venues while ensuring player fitness and minimizing travel fatigue is a recurring challenge.

This project aims to analyze IPL data to uncover patterns, trends, and insights that can help improve decision-making for teams, organizers, and stakeholders. By addressing these challenges, the project seeks to contribute to a deeper understanding of how sports analytics and management can enhance the overall IPL ecosystem.

DATA COLLECTION:

```
driver = webdriver.Chrome()
```

```
driver.get("https://www.iplt20.com/stats/2025")


# Scroll multiple times to trigger lazy loading
for i in range(10):  # adjust until all rows are loaded
    driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
    time.sleep(2)  # wait for Angular to fetch and render


soup = BeautifulSoup(driver.page_source, "html.parser")
table = soup.find("table", class_="st-table statsTable ng-scope")
headers=table.find_all("th")
head=[]
for i in headers:
    heads=i.text
    head.append(heads)
rows = []
for row in table.find_all("tr")[1:]:
    cols = [td.get_text(strip=True) for td in row.find_all("td")]
    if cols:
        rows.append(cols)
print(len(rows))
driver.quit()
```

for data scraping we used beautiful soup and selenium .We created a chrome driver after opening that we provided the link for the website we want to scrap so the website can be automatically opened. We used lazy loading for more optimized results and improved performance. After that we used soup to read the html and find the table we need using `table = soup.find("table", class_="st-table statsTable ng-scope").` After finding the table we have to find the head and the column rows and finally we printed the length of the rows we have scraped and quit the driver.

In our case we scrapped ipl data of players with columns as Player, Runs, Match, Innings, Not Outs, Highest score, Average, Balls faced, Strike Rate, 100,50,4s, 6s.

The number of rows is 156

```
df=pd.DataFrame(rows,columns=head)
```

```
df.head(150)
```

we created a data frame of our data using pandas and after removing unnecessary columns we saved the data to csv file using

```
df2.to_csv("ipl auction.csv")
```

DATA PREPROCESSING:

```
df2.shape
```

we got that there are total of 156 rows and 13 columns in our data

```
df2.describe()
```

we got basic statistics like mean, median, std, and IQR

```
df2.duplicated().sum()
```

we got duplicated values as 0

UNIVARIATE ANALYSIS:

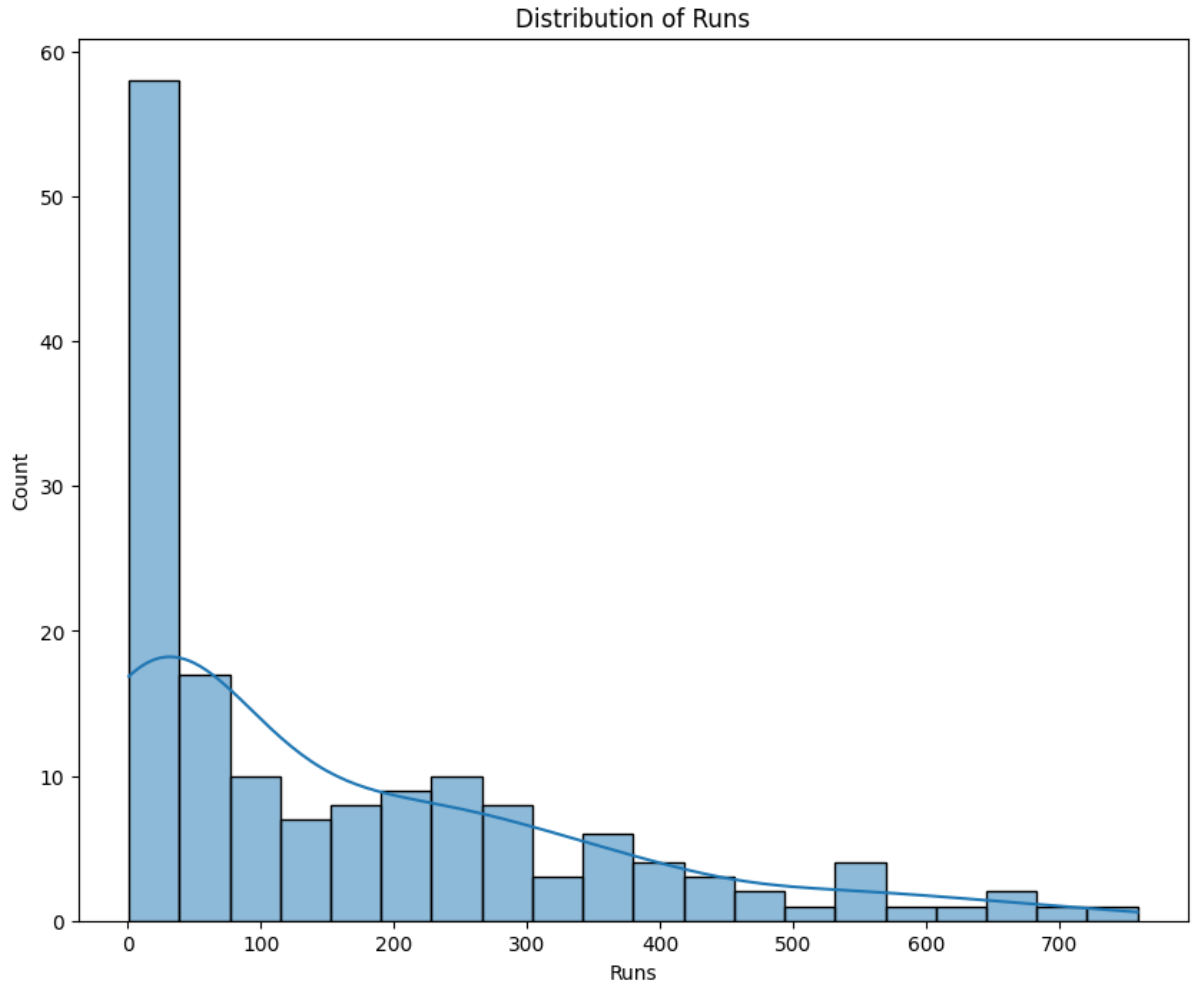
first we will create a variable named numeric_col which only have numeric values for that;

```
numeric_cols=df2.drop(columns="Player")
```

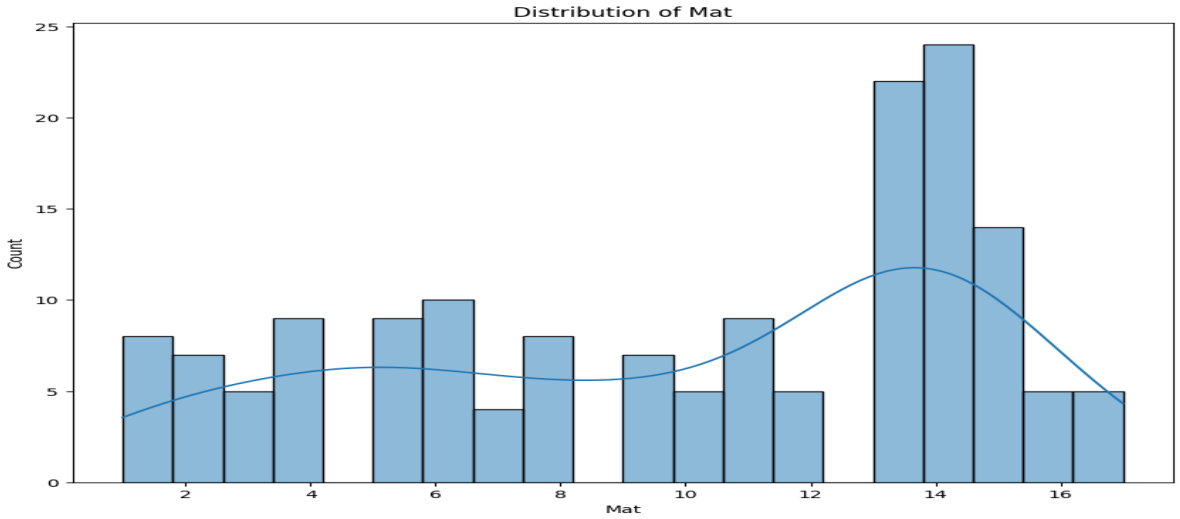
```
numeric_cols
```

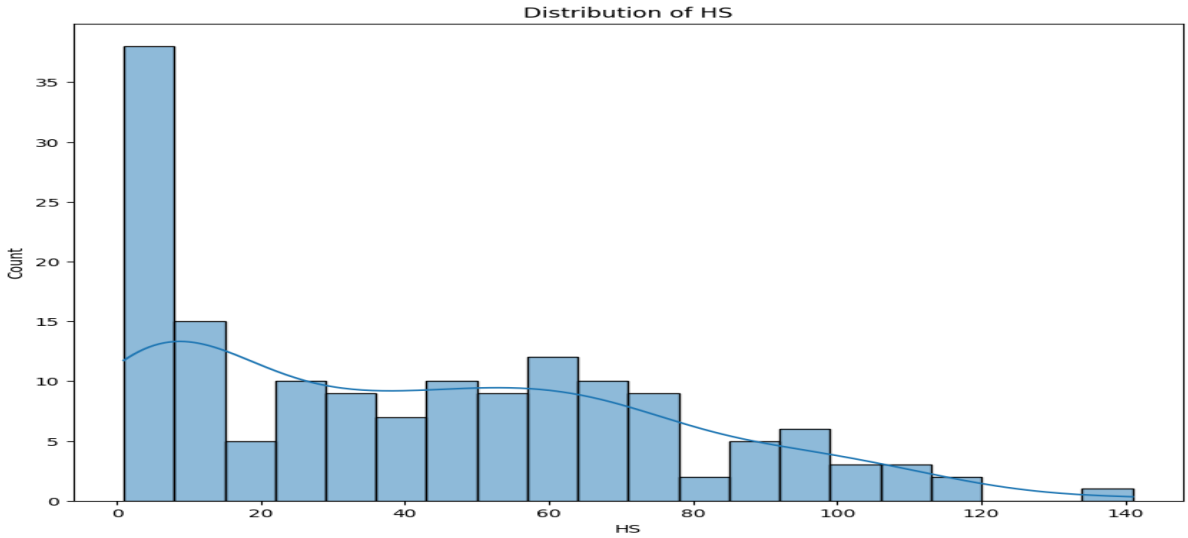
now lets plot some graphs

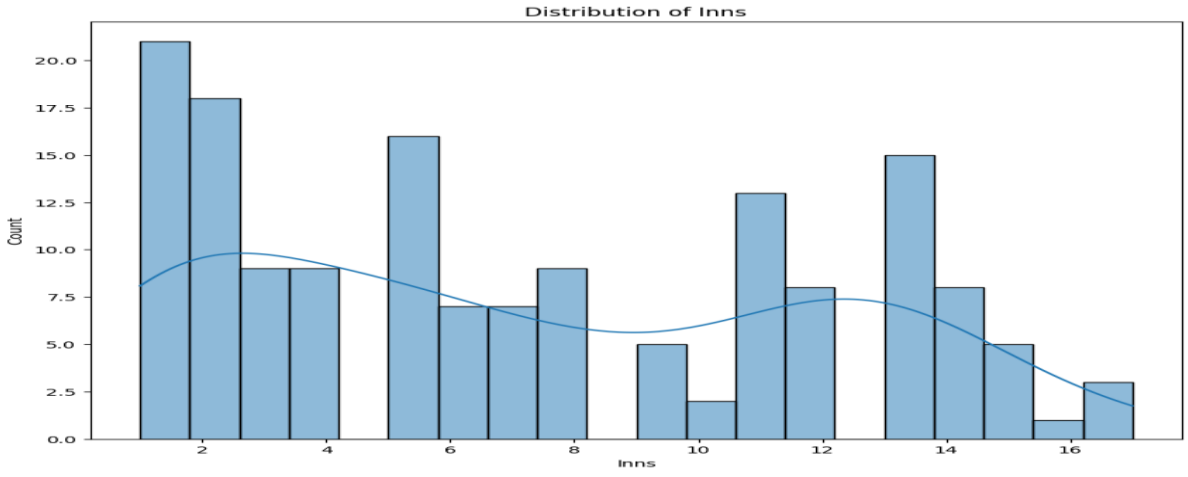
```
for col in numeric_cols:
    plt.figure(figsize=(10,8))
    sns.histplot(df2[col],bins=20,kde=True)
    plt.title(f"Distribution of {col}")
    plt.show()
```

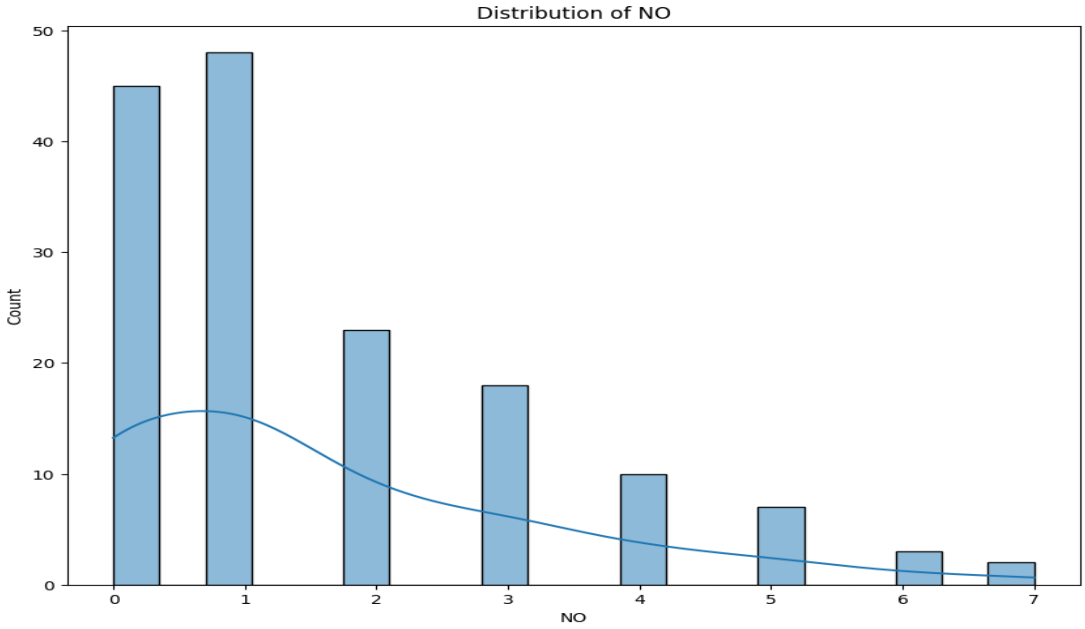


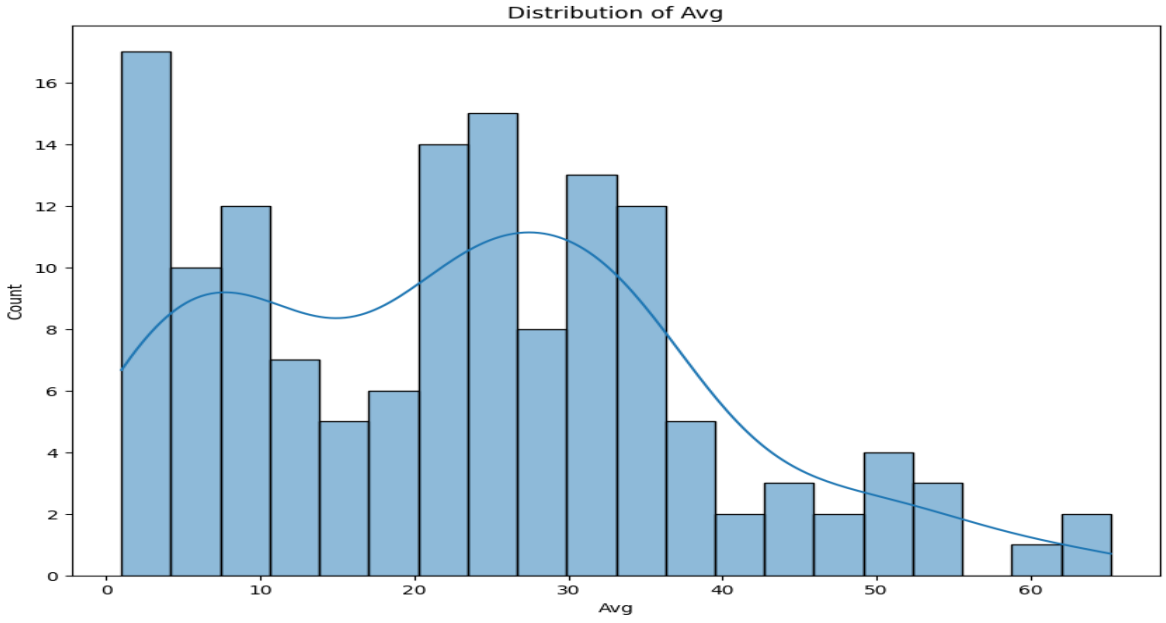
This is the distribution of runs we can see that most players scored between 0 and 100 and its gradually decreasing indicating few excellent performing players. this can also vary according to features like mat, inns and balls faced.


Distribution of Mat
Count
Mat
0
5
10
15
20
25
2
4
6
8
10
12
14
16

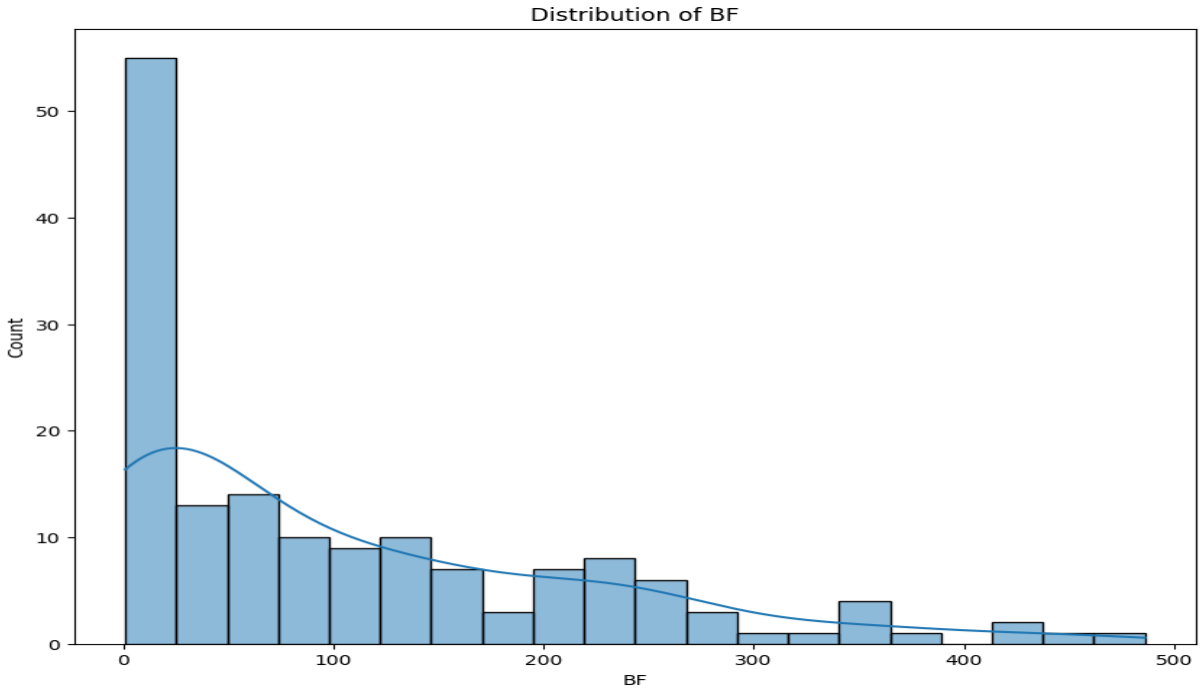

Distribution of HS
Count
HS
0
5
10
15
20
25
30
35
0
20
40
60
80
100
120
140

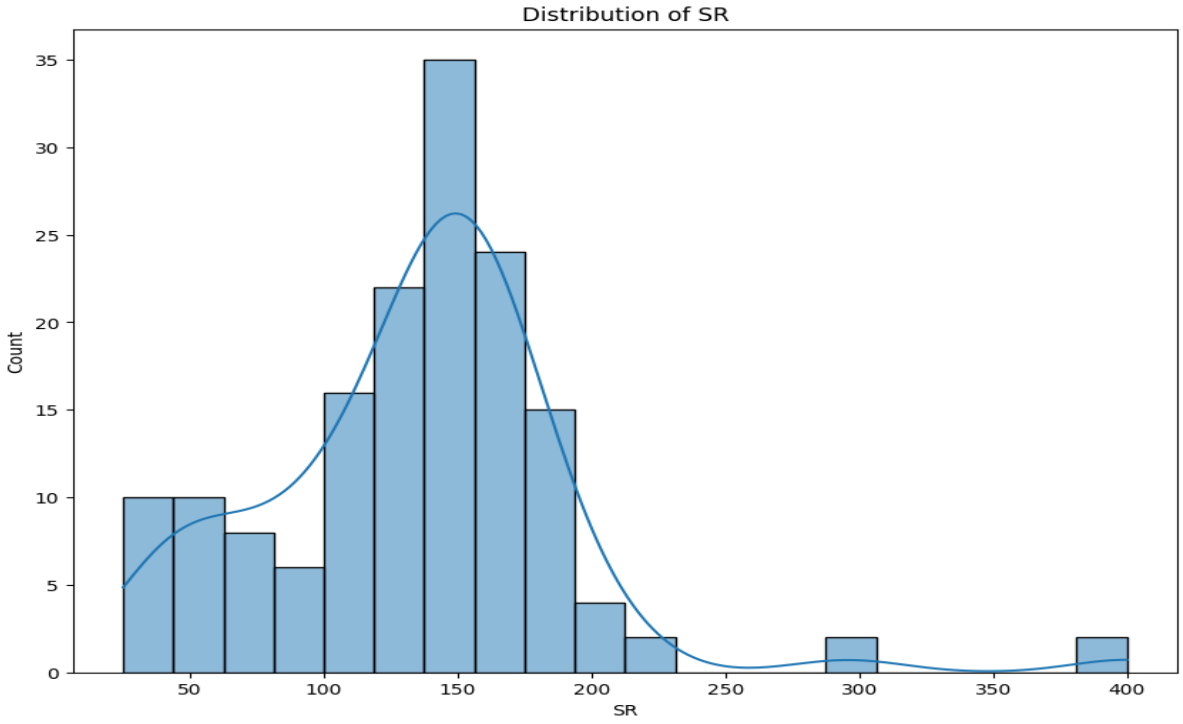

Distribution of Inns
Count
Inns
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
2
4
6
8
10
12
14
16



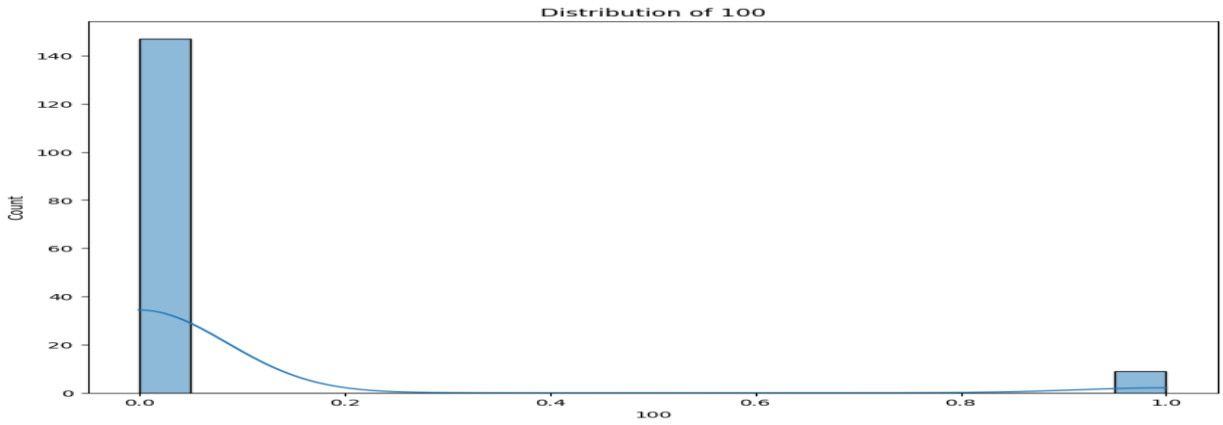


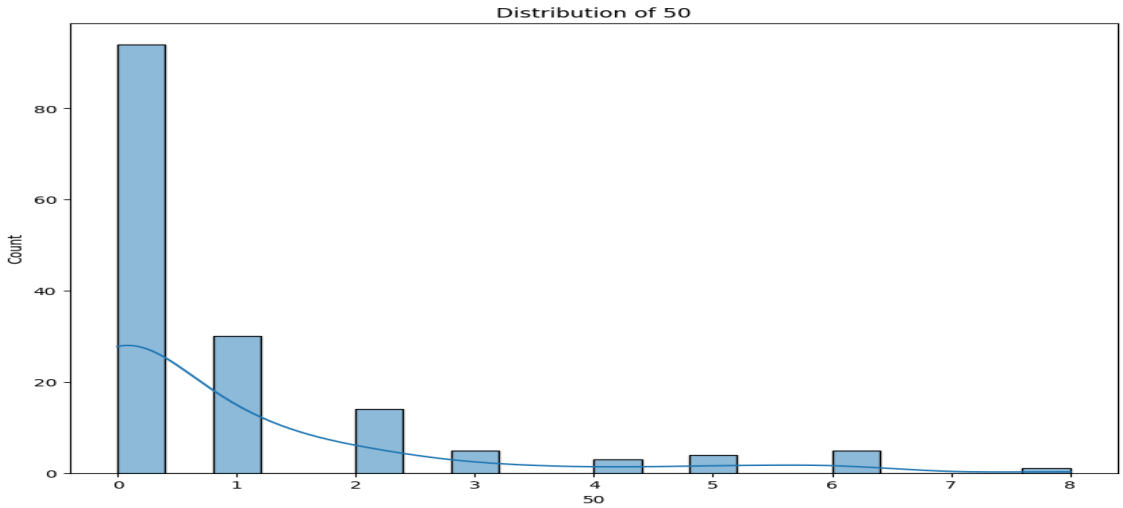
In the distribution of avg it is same as runs the most excellent players score more avg runs

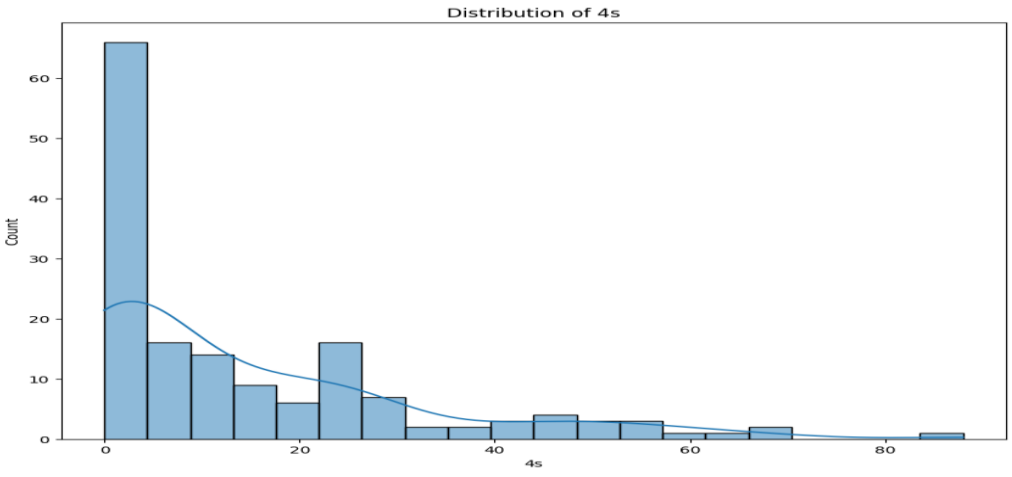


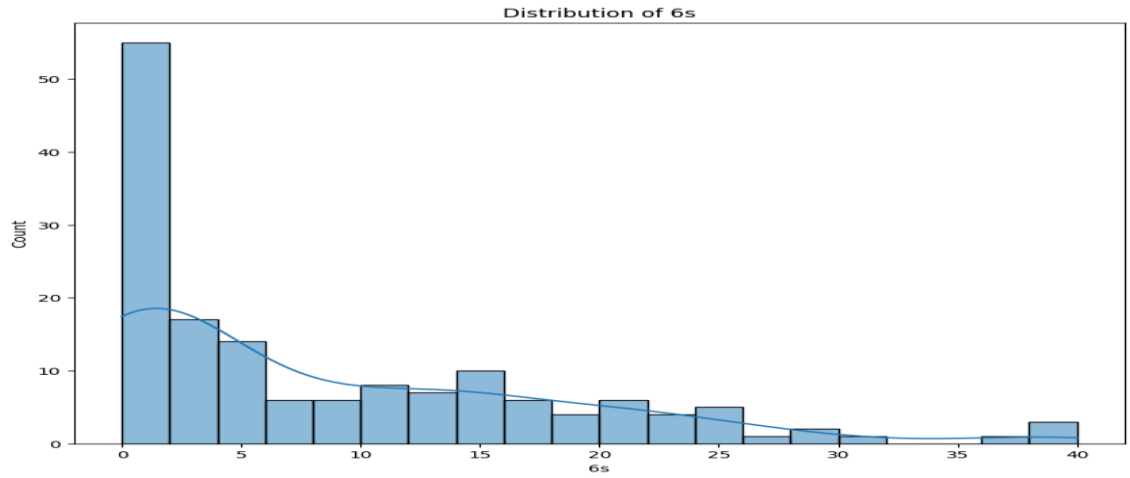


In this graph we can see that the striking rate is very high for some players it may be because the ball faced is very high for players who played less innings.


Distribution of 100
Count
100
0
20
40
60
80
100
120
140
0.0
0.2
0.4
0.6
0.8
1.0


Distribution of 50
Count
50
0
20
40
60
80
0
1
2
3
4
5
6
7
8

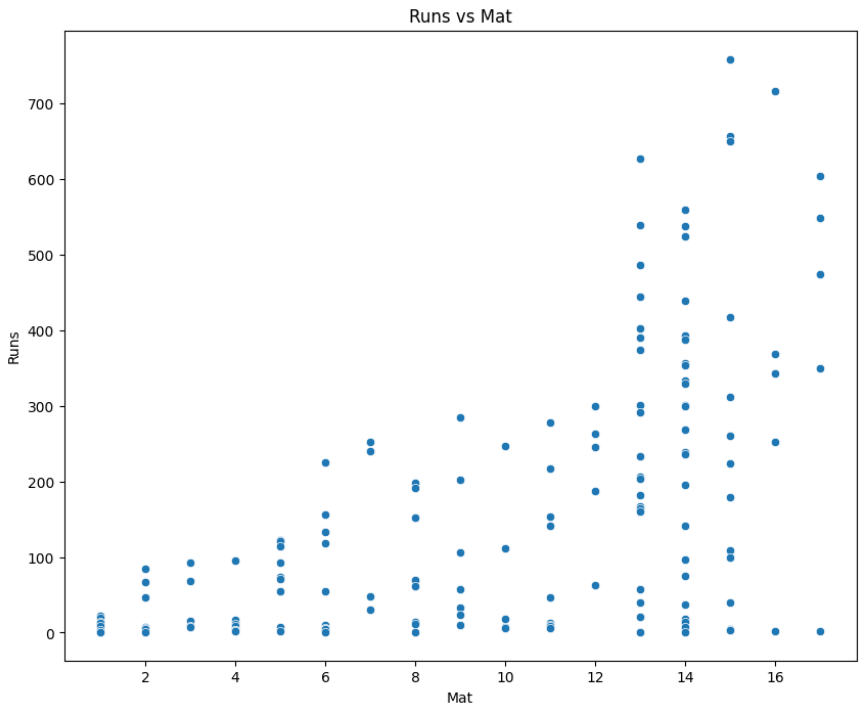

Distribution of 4s
Count
4s
0
10
20
30
40
50
60
0
20
40
60
80



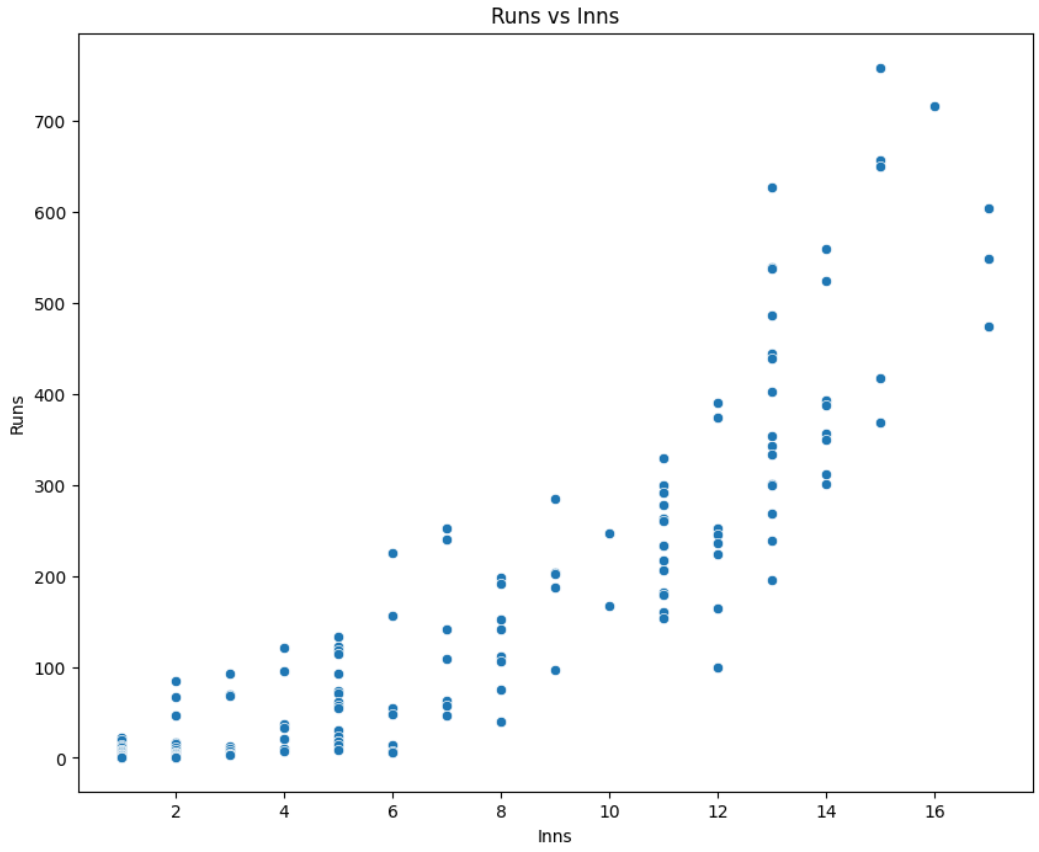
From these plots we understand the distribution of different attributes of players.

BIVARIATE:

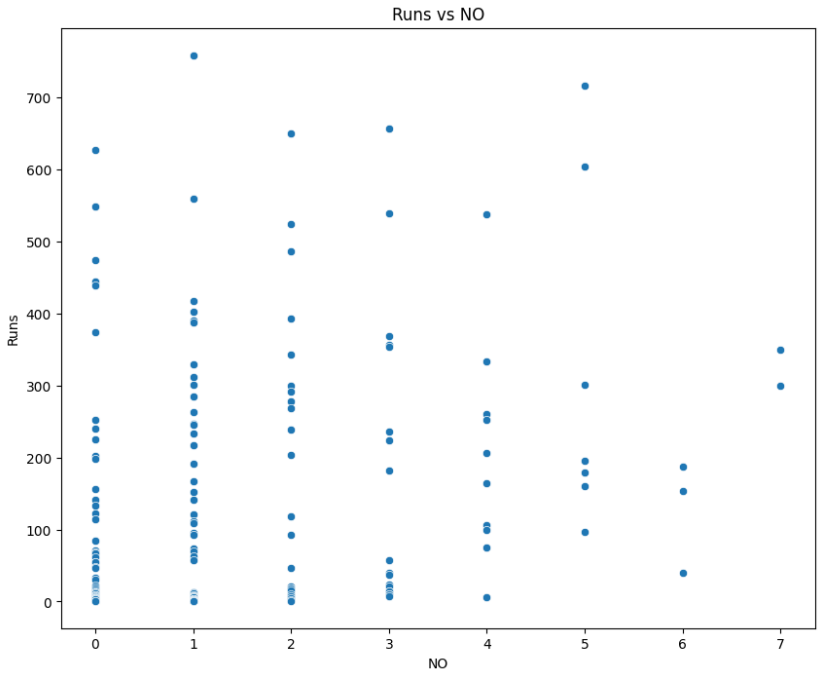
```
for col in numeric_cols:
    plt.figure(figsize=(10,8))
    sns.scatterplot(x=col,y="Runs",data=df2)
    plt.title("Runs vs "+col)
    plt.show
```

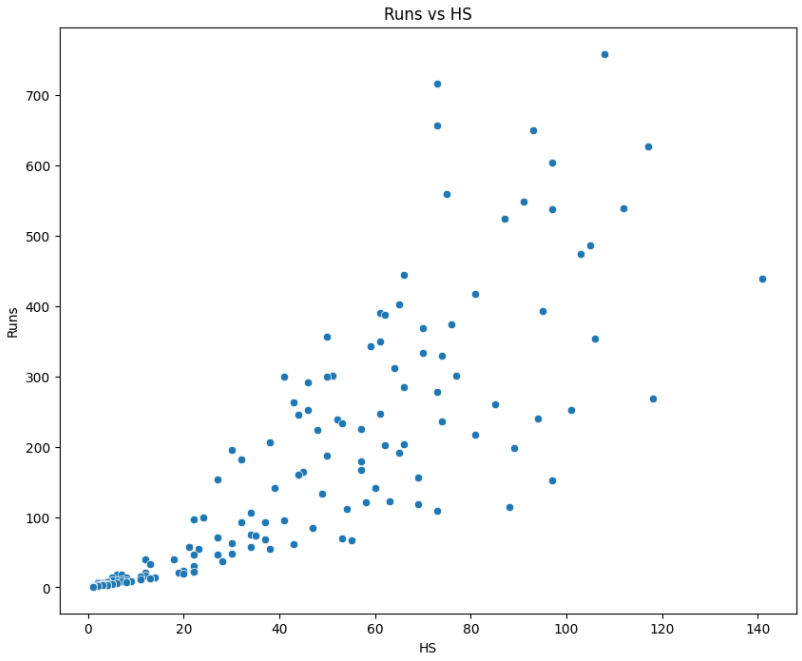


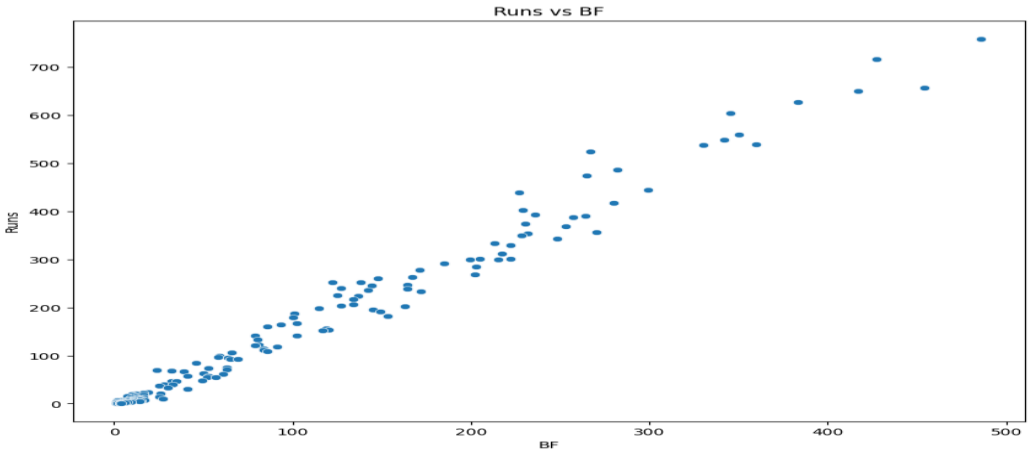
In run vs mat we can see that runs increases as number of matches played increases

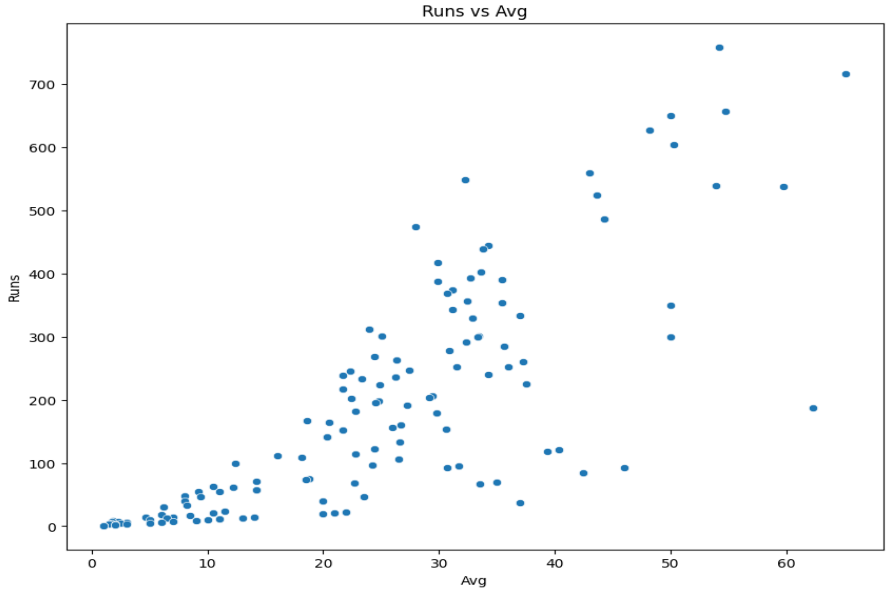


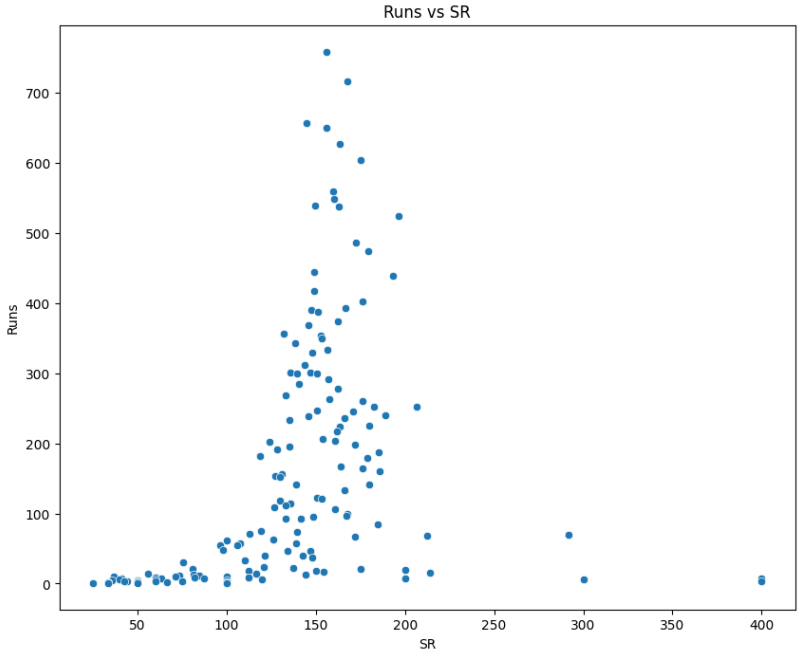
For innings also runs increases over time



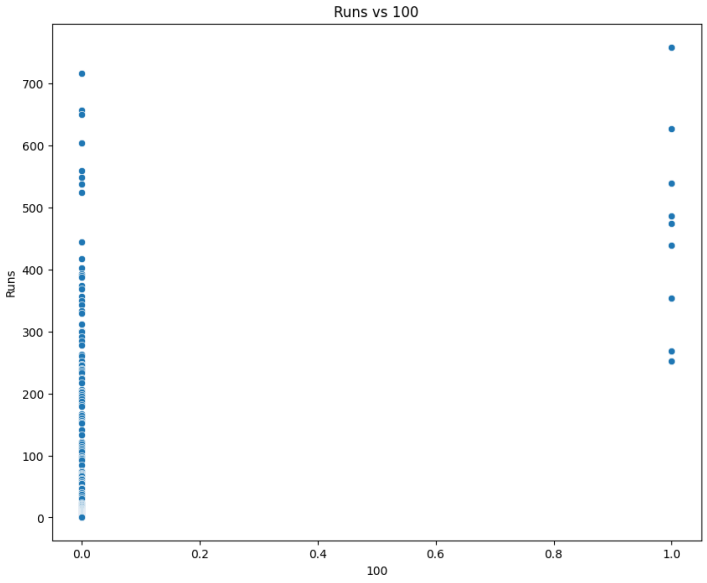

Runs vs HS
Runs
HS
0
100
200
300
400
500
600
700
0
20
40
60
80
100
120
140


Runs vs BF
Runs
BF
0
100
200
300
400
500
600
700

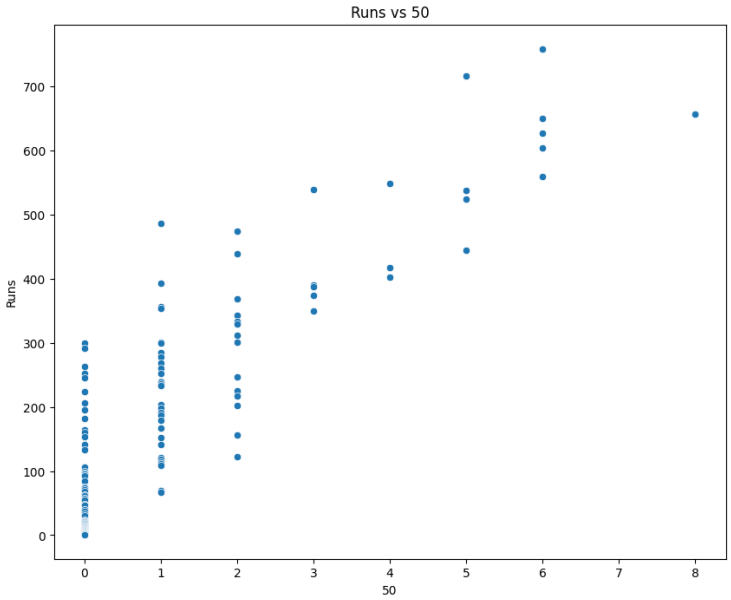

Runs vs Avg
Runs
Avg
0
100
200
300
400
500
600
700
0
10
20
30
40
50
60



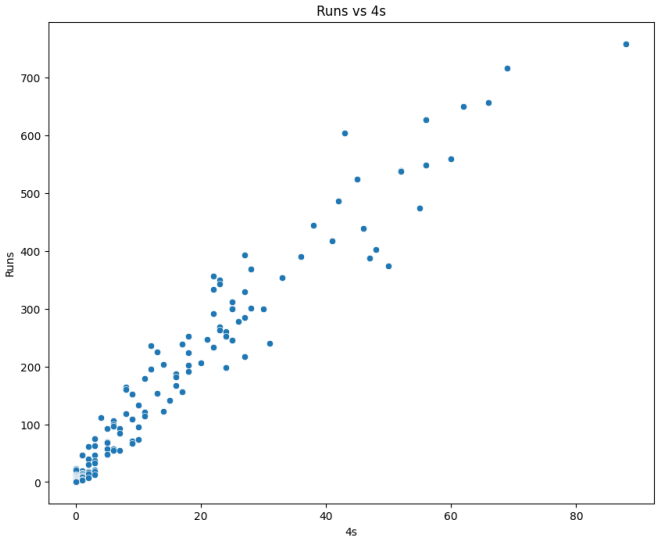
Most of the graph shows linear relations between runs and variables but here for SR it increases tremendously for range 100 and 200 and less runs for strike rate of 400. This can be due to many factors such as less number of matches played, the number of balls faced is very low, innings is very low that's why the player with strike rate 400 have very low runs while the players with 100-200 strike rate have higher runs

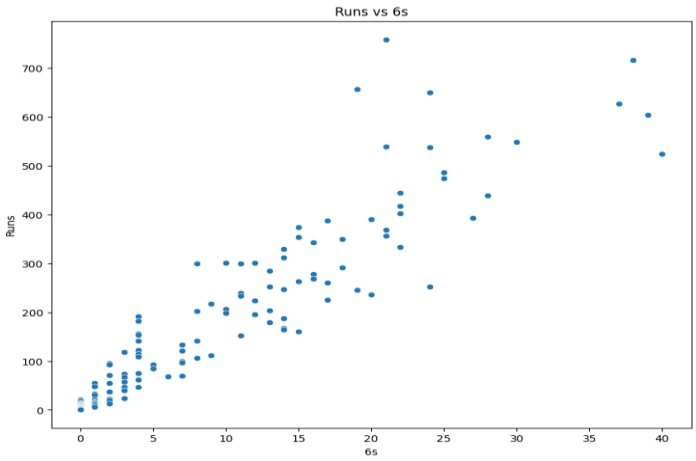


There are only a few number of players who crossed century.

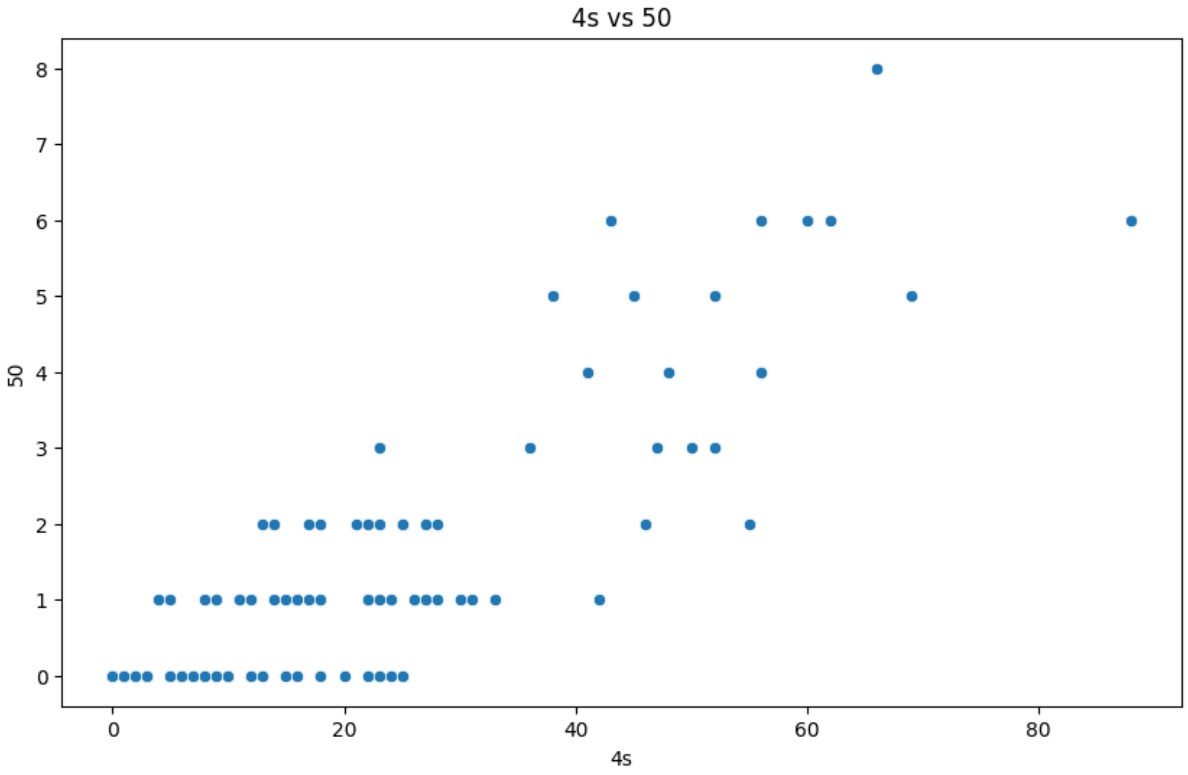


When number of 50s increases the total runs increases.



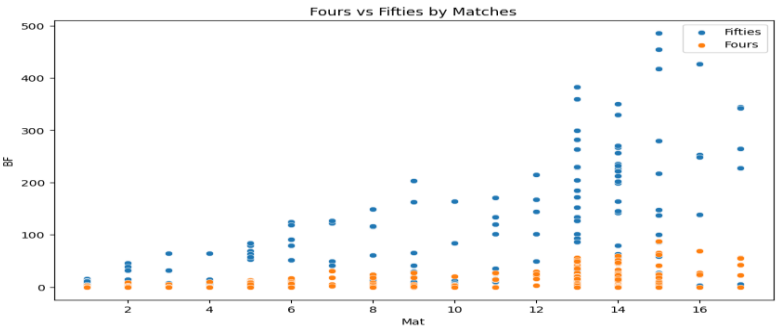


```
plt.figure(figsize=(10,6))
sns.scatterplot(x="4s",y="50",data=df2)
plt.title("4s vs 50")
plt.show()
```



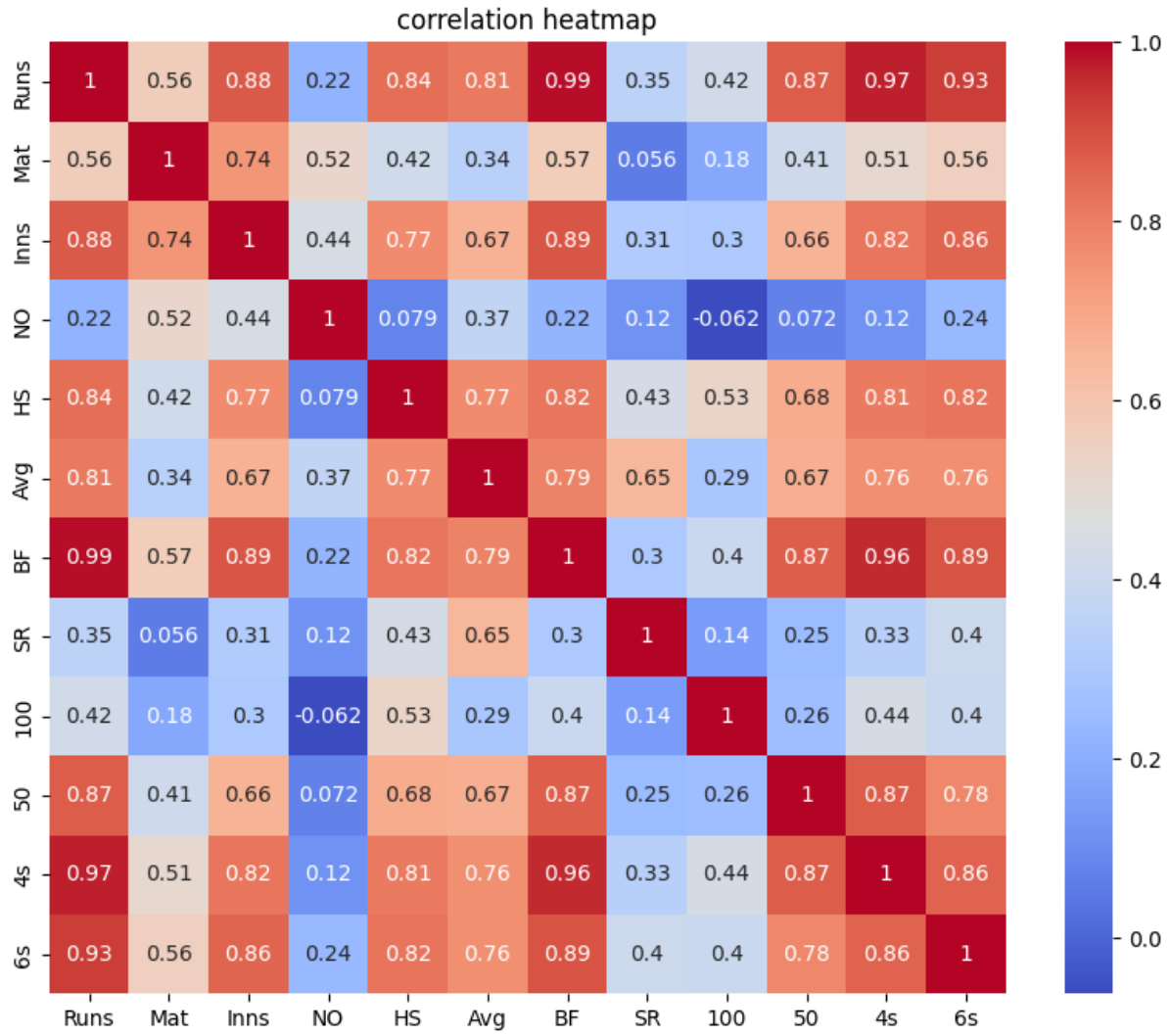
This is the relationship between 4s and 50s. when the number of fours increases the 50 is also increases

```
plt.figure(figsize=(10,6))
sns.scatterplot(x="Mat", y="BF", data=df2, label="Fifties")
sns.scatterplot(x="Mat", y="4s", data=df2, label="Fours")
plt.title("Fours vs Fifties by Matches")
plt.legend()
plt.show()
```



MULTIVARIATE:

```
plt.figure(figsize=(10,8))
sns.heatmap(numeric_cols.corr(),annot=True,cmap="coolwarm")
plt.title("correlation heatmap")
plt.show()
```



- Runs vs BF- Extremely strong correlation (0.99). This makes sense—more balls faced usually means more runs scored.
- Run vs Boundaries- Runs have higher correlation with both 4s and 6s. Boundary hitting is a major driver of run accumulation.
- Run vs Avg- Strong correlation (0.81). Players who score more runs tend to maintain higher batting averages
- Runs vs Strike Rate(SR)- It have a moderate correlation (0.35). This shows that scoring big runs doesn't necessarily mean scoring quickly—accumulation and acceleration are somewhat independent.

- 50 have higher correlations with 4s,6s and BF when 50s increases the number of ball faced and number of 4s and 6s will increase.
- centuries- surprisingly Runs have weaker correlation with 100(0.47) than 50 (0.87). This suggests consistency (frequent 50s) is more strongly tied to overall run tally than occasional big scores.
- NO- Weak correlations across most metrics. Being not out doesn't strongly influence overall performance indicators except a mild effect on average (0.37).
- Matches- Only moderately correlated with Runs (0.56). Simply playing more matches doesn't guarantee higher run totals—it depends on innings played and performance quality.

MODEL BUILDING:

In this we are going to predict a players runs based on features so we will take target as runs and we are going to build regression model to predict runs.
we will take mainly 4 models to train and find the best parameter and hyperparameter using optuna. after that we are going to log our metrics into mlflow for experiment tracking and for future reference.

```
x=numeric_cols.drop("Runs",axis=1)

y=df2["Runs"]

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.3,random_state=42)

scaler=StandardScaler()

x_train_scaled=scaler.fit_transform(x_train)

x_test_scaled=scaler.fit_transform(x_test)
```

we split our data into x and y using train test split method after that we applied standard scaler to make every feature values to have similar range.

```
def objective(trial):
    model_name = trial.suggest_categorical("model",
                                    ["LinearRegression",
                                     "KNeighborsRegressor",
                                     "DecisionTreeRegressor",
                                     "GradientBoostingRegressor"])

    if model_name == "LinearRegression":
        estimator = LinearRegression(
            fit_intercept=trial.suggest_categorical("lr_fit_intercept", [True, False]),
            positive=trial.suggest_categorical("lr_positive", [True, False])
        )
```

```
        rfecv = RFECV(estimator=estimator, cv=5, scoring="r2", n_jobs=-1)
        pipeline = Pipeline([
            ("imputer", SimpleImputer(strategy="mean")),
            ("feature_selection", rfecv),
            ("estimator", estimator)
        ])

    elif model_name == "KNeighborsRegressor":
        estimator = KNeighborsRegressor(
            n_neighbors=trial.suggest_int("knn_n_neighbors", 3, 11, step=2),
            weights=trial.suggest_categorical("knn_weights", ["uniform", "distance"])
        )
        pipeline = Pipeline([
            ("imputer", SimpleImputer(strategy="mean")),
            ("estimator", estimator)
        ])

    elif model_name == "DecisionTreeRegressor":
        estimator = DecisionTreeRegressor(
            random_state=42,
            max_depth=trial.suggest_categorical("dt_max_depth", [None, 5, 10]),
            min_samples_split=trial.suggest_int("dt_min_samples_split", 2, 5),
            min_samples_leaf=trial.suggest_int("dt_min_samples_leaf", 1, 2)
        )
        rfecv = RFECV(estimator=estimator, cv=5, scoring="r2", n_jobs=-1)
        pipeline = Pipeline([
            ("imputer", SimpleImputer(strategy="mean")),
            ("feature_selection", rfecv),
            ("estimator", estimator)
        ])
```

```
    elif model_name == "GradientBoostingRegressor":
        estimator = GradientBoostingRegressor(
            random_state=42,
            n_estimators=trial.suggest_categorical("gb_n_estimators", [100, 200]),
            learning_rate=trial.suggest_float("gb_learning_rate", 0.05, 0.1),
            max_depth=trial.suggest_categorical("gb_max_depth", [3, 5])
        )
        rfecv = RFECV(estimator=estimator, cv=5, scoring="r2", n_jobs=-1)
        pipeline = Pipeline([
            ("imputer", SimpleImputer(strategy="mean")),
            ("feature_selection", rfecv),
            ("estimator", estimator)
        ])

    score = cross_val_score(pipeline, x_train_scaled, y_train, cv=5, scoring="r2").mean()
    return score
study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=30)

best_params = study.best_params
best_score = study.best_value

with mlflow.start_run(run_name="Optuna_Best_Run"):
    mlflow.log_param("best_model", best_params["model"])

    # Log each parameter with a clear prefix
    for k, v in best_params.items():
        mlflow.log_param(f"optuna_{k}", v)

    mlflow.log_metric("best_cv_r2", best_score)
print("Best Model:", best_params["model"])
```

```
print("Best CV R2 Score:", best_score)
```

in here firstly we define a function with parameter trial this is how optuna evaluate each model with different parameters. then we named our four models which are linear, knn, decision tree and gb.

By using nested if we make trial for each model with respective parameter optuna will go through all of the trials and find the best model and its r2 value.

After that we include each model in a pipeline which include
imputer, optimal rfecv and respective estimator

We use cross validation for scoring fold id five and it will return the mean value of all 5 folds.

Then we created study in which optuna will try to maximize the r2 value. optuna will run 30 trials each with different hyperparameter

After that we logged the best model and r2 value in to mlflow using

Mlflow.log_param and mlflow.log_metric

For the final model we will retrain the data on full training set

```
# Refit the best model on full training data
if best_params["model"] == "LinearRegression":
    best_model = LinearRegression(
        fit_intercept=best_params["lr_fit_intercept"],
        positive=best_params["lr_positive"]
    )
elif best_params["model"] == "KNeighborsRegressor":
    best_model = KNeighborsRegressor(
        n_neighbors=best_params["knn_n_neighbors"],
        weights=best_params["knn_weights"]
    )
elif best_params["model"] == "DecisionTreeRegressor":
    best_model = DecisionTreeRegressor(
        random_state=42,
        max_depth=best_params["dt_max_depth"],
        min_samples_split=best_params["dt_min_samples_split"],
        min_samples_leaf=best_params["dt_min_samples_leaf"]
    )
```

```
elif best_params["model"] == "GradientBoostingRegressor":
    best_model = GradientBoostingRegressor(
        random_state=42,
        n_estimators=best_params["gb_n_estimators"],
        learning_rate=best_params["gb_learning_rate"],
        max_depth=best_params["gb_max_depth"]
    )

pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("estimator", best_model)
])

# Fit on full training data
pipeline.fit(x_train_scaled, y_train)

# Log the trained pipeline into MLflow
with mlflow.start_run(run_name="Optuna_Best_Model_Save"):
    mlflow.log_param("best_model", best_params["model"])
    for k, v in best_params.items():
        mlflow.log_param(f"optuna_{k}", v)
    mlflow.log_metric("best_cv_r2", best_score)
    mlflow.sklearn.log_model(pipeline, "optuna_best_model")
```

we logged the data into mlflow successfully if we check the mlflow we will see the final best model and best r2 value.

Now we will try to train the data on linear regression which is the best model to form pkl file.

```
features = df2[["Inns", "NO", "Avg", "BF", "4s", "6s"]]
imputer=SimpleImputer(strategy="mean")
x=imputer.fit_transform(features)
```

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.3,random_state=42)

x_train_scal=scaler.fit_transform(x_train)

x_test_scal=scaler.fit_transform(x_test)

model=LinearRegression()

model.fit(x_train_scal,y_train)

y_pred=model.predict(x_test_scal)

print("R² Score:", r2_score(y_test, y_pred))

print("MSE:", mean_squared_error(y_test, y_pred))
```

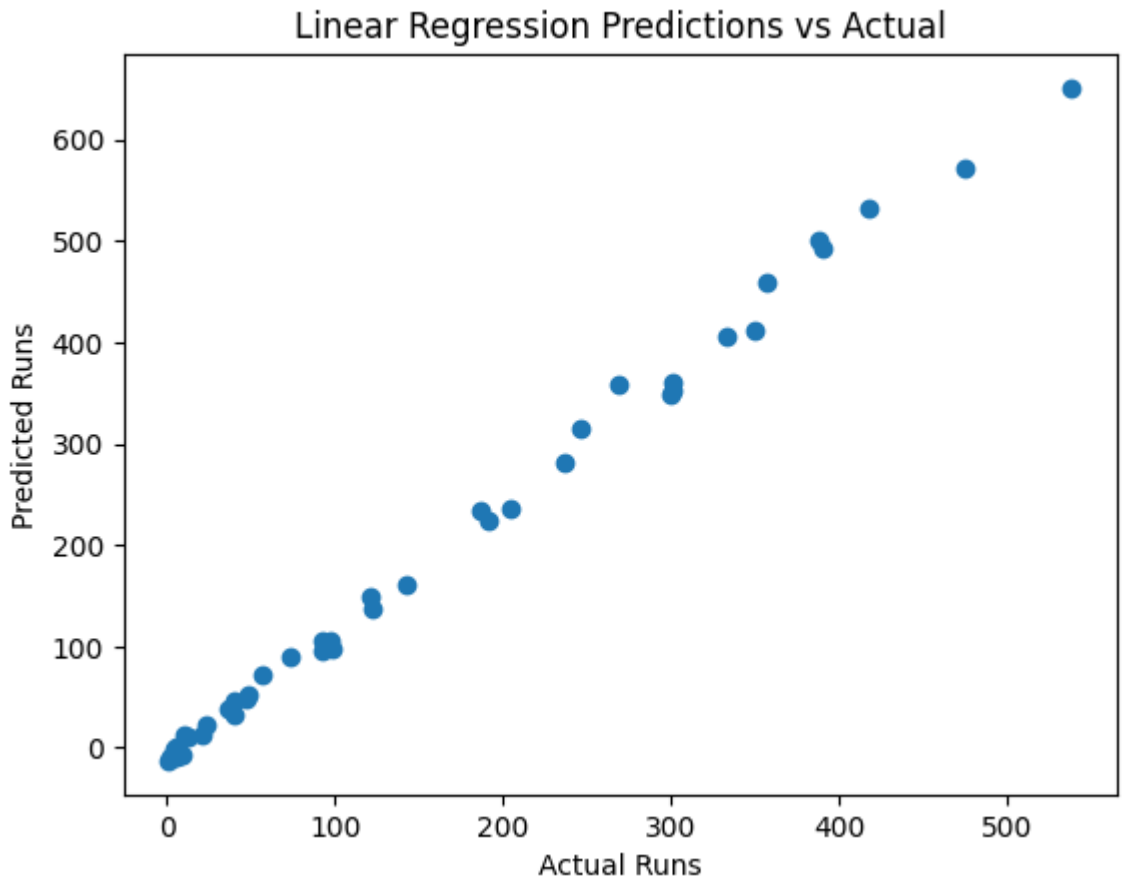
R^2 Score: 0.897227844124386

MSE: 2339.0853350653574

The r2 value we optained is lesser than before this may because of overfitting and underfitting.we can plot residual plot for further analysis.

PREDICTED VS ACTUAL:

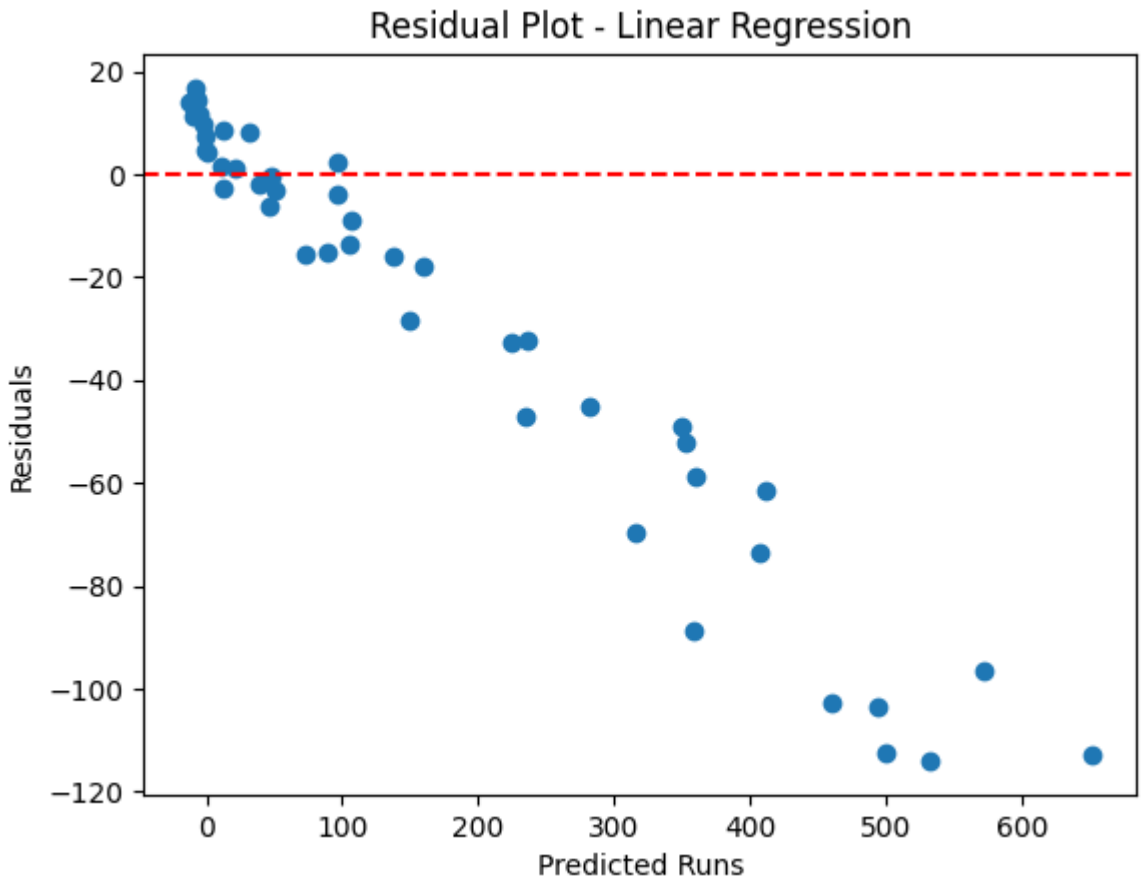
```
plt.scatter(y_test, y_pred)

plt.xlabel("Actual Runs")

plt.ylabel("Predicted Runs")

plt.title("Linear Regression Predictions vs Actual")

plt.show()
```



In this plot we can see that the predicted and actual values are in a diagonal line which indicates model is capturing the overall relationship very well.

RESIDUAL PLOT:

```
residuals = y_test - y_pred

plt.scatter(y_pred, residuals)

plt.axhline(0, color='red', linestyle='--')

plt.xlabel("Predicted Runs")

plt.ylabel("Residuals")

plt.title("Residual Plot - Linear Regression")

plt.show()
```



We can see a downward pattern in the graph. normally if the model captures all relationship well the residuals should be scattered around 0. Which means our model may be too simple to capture the relationship well. May be we can use more complex models like polynomial regression for this.

```
# Select features and target
x = df2[["Inns", "NO", "Avg", "BF", "4s", "6s"]]
y = df2["Runs"]
imputer = SimpleImputer(strategy="mean")
x_imputed = imputer.fit_transform(x)
x_train, x_test, y_train, y_test = train_test_split(x_imputed, y, test_size=0.3, random_state=42)
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
poly = PolynomialFeatures(degree=2, include_bias=False)
x_train_poly = poly.fit_transform(x_train_scaled)
x_test_poly = poly.transform(x_test_scaled)
```

```
model = LinearRegression()
model.fit(x_train_poly, y_train)
y_pred_poly = model.predict(x_test_poly)

print("Poly R²:", r2_score(y_test, y_pred_poly))
print("Poly MSE:", mean_squared_error(y_test, y_pred_poly))
```

Poly R²: 0.9998316992485593

Poly MSE: 3.8305104745679315

After fitting polynomial we can see that the r2 value increased drastically.

Now we will use this final model for everything.

```
# training set
y_pred_train=model.predict(x_train_poly)
r2=r2_score(y_train,y_pred_train)
mse_train=mean_squared_error(y_train,y_pred_train)
#testing set
y_pred_test= model.predict(x_test_poly)
r2_test= r2_score(y_test,y_pred_test)
mse_test=mean_squared_error(y_test,y_pred_test)

print("score of training set:",r2)
print("mse_train:",mse_train)
print("score of test:",r2_test)
print("mse_test:",mse_test)
```

score of training set: 0.9999399601045433

mse_train: 2.1754908376578435

score of test: 0.9998316992485593

mse_test: 3.8305104745679315

in here we can see that both r2 values are similar for training and testing.

We will log this model into mlflow

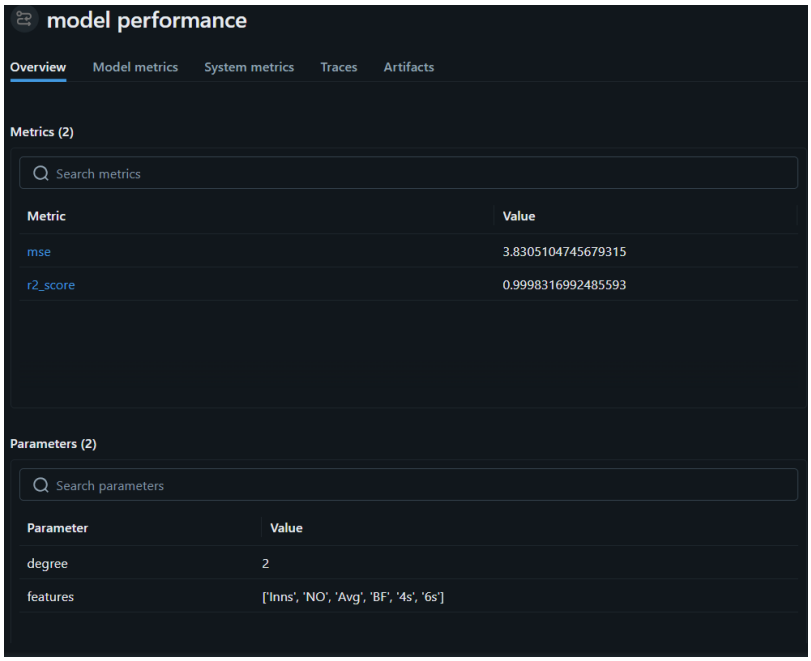
```
import mlflow
import mlflow.sklearn
```

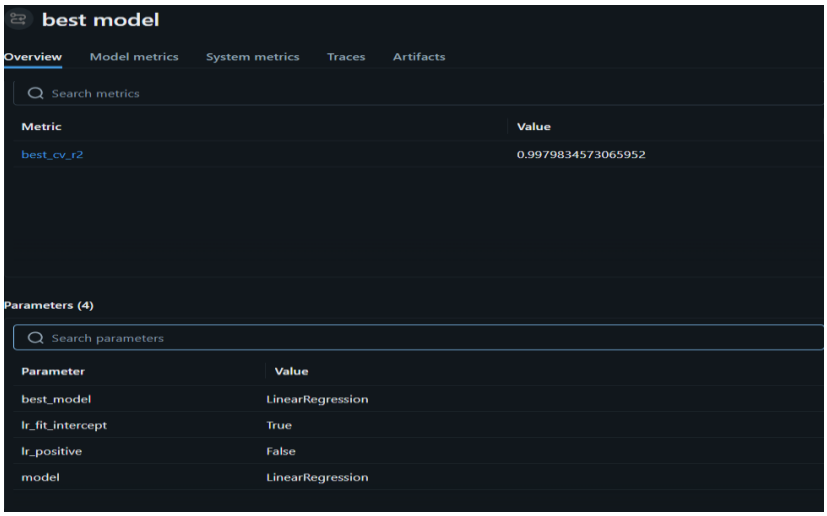
```
with mlflow.start_run():
    model.fit(x_train_poly, y_train)
    y_pred_poly = model.predict(x_test_poly)

    r2 = r2_score(y_test, y_pred_poly)
    mse = mean_squared_error(y_test, y_pred_poly)

    mlflow.log_param("degree", 2)
    mlflow.log_param("features", ["Inns", "NO", "Avg", "BF", "4s", "6s"])
    mlflow.log_metric("r2_score", r2)
    mlflow.log_metric("mse", mse)

    mlflow.sklearn.log_model(model, "ipl_model")
```

now we will form our pkl file for fast api

```
# Define pipeline
poly_lr_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("scaler", StandardScaler()),
    ("poly", PolynomialFeatures(degree=2, include_bias=False)),
    ("model", LinearRegression())
])
x = df2[["Inns", "NO", "Avg", "BF", "4s", "6s"]]
y = df2["Runs"]
poly_lr_pipeline.fit(x, y)
# Save pipeline
joblib.dump(poly_lr_pipeline, "poly_linear_regression_pipeline.pkl")
```

EXPERIMENT TRACKING MLFLOW:





This is the pictures of our model performance from mlflow. And the log of best model we saved by optuna.

In this it is trackes as the best model is linear regression with

Fit intercept—true

Positive—false

And model performance is excellent with

R2=0.9998

Mse=3.83

MODEL DEPLOYMENT:

We are going to create a backend and frontend for our data for backend we will use fastapi and for frontend we will use streamlit.

Fastapi:

```
import os
import logging
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
import joblib
import pandas as pd
from prometheus_client import Gauge
from prometheus_fastapi_instrumentator import Instrumentator


# Configure logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("ipl_backend")


app = FastAPI(title="IPL Runs Prediction", version="1.0")


# Load trained model once at startup, with error handling
try:
    model = joblib.load("model/poly_linear_regression_pipeline.pkl")
```

```
    logger.info("Model loaded successfully.")
except Exception as e:
    logger.error(f"Model failed to load: {e}")
    raise RuntimeError("Model could not be loaded.")

# Prometheus instrumentation
Instrumentator().instrument(app).expose(app)

# Custom metric for leakage gap
leakage_metric = Gauge("data_leakage_gap", "Gap between train and test R2")

def log_leakage_gap(train_score: float, test_score: float):
    """Log real leakage gap from training pipeline."""
    gap = train_score - test_score
    leakage_metric.set(gap)
    logger.info(f"Leakage gap logged: {gap}")

class PlayerData(BaseModel):
    Inns: int
    NO: int
    Avg: float
    BF: int
    four_runs: int = Field(..., alias="4s")
    six_runs: int = Field(..., alias="6s")

@app.get("/")
def root():
    return {"message": "IPL API is running"}

@app.post("/predict")
def predict_runs(player: PlayerData):
```

```
    try:
        df = pd.DataFrame([player.dict(by_alias=True)])
        prediction = model.predict(df)[0]
        return {"Predicted Runs": round(float(prediction), 0)}
    except Exception as e:
        logger.error(f"Prediction failed: {e}")
        raise HTTPException(status_code=500, detail="Prediction error")

# Only enable simulation in development mode
if os.getenv("DEBUG", "false").lower() == "true":
    @app.get("/simulate_leakage/{gap_value}")
    def simulate_leakage(gap_value: float):
        leakage_metric.set(gap_value)
        logger.warning(f"Simulated leakage gap set to {gap_value}")
        return {"message": f"Leakage gap set to {gap_value}"}
```

we are important necessary libraries including os and logging. Os is used to read env in our project and logging is used to log everything in our data like the errors occurred, info, etc.

we have created env where if DEBUG is true then we can stimulate leakage by default the DEBUG is false. We have defined logging for different things like for info, for errors for different scenarios.

STREAMLIT:

```
import streamlit as st
import requests

st.title("IPL Player Runs Prediction")

# Input fields
Inns = st.number_input("Innings", min_value=0)
NO = st.number_input("Not Outs", min_value=0)
Avg = st.number_input("Average", min_value=0.0)
```

```
BF = st.number_input("Balls Faced", min_value=0)
Fours = st.number_input("Number of 4s", min_value=0)
Sixes = st.number_input("Number of 6s", min_value=0)

# Use FastAPI service name when running inside Docker Compose
FASTAPI_URL = "http://fastapi:8000/predict"

if st.button("Predict Runs"):
    try:
        response = requests.post(
            FASTAPI_URL,
            json={
                "Inns": Inns,
                "NO": NO,
                "Avg": Avg,
                "BF": BF,
                "4s": Fours,
                "6s": Sixes
            }
        )
        if response.status_code == 200:
            result = response.json()
            st.success(f"Predicted Runs: {result['Predicted Runs']:.2f}")
        else:
            st.error(f"Error {response.status_code}: {response.text}")
    except requests.exceptions.ConnectionError:
        st.error("Could not connect to FastAPI")
```

imported libraries and put the name of streamlit app as IPL PLAYER RUN PREDICTION.

By st.number_input we are creating a box to put feature values. Users need to enter the stats inside this box to get prediction.

Then it gives the stats to the backend and fetch the result from fastapi.

So when we click predict runs streamlit sends a post request to fastapi. requests contains player stats in jsons format.

If fastapi responds with 200 the prediction will be displayed in the ui with st.success. if theres an error it will show could not connect to fast api.

MONITORING AND RULES:

We have successfully created a backend and frontend for our model. Now we have to monitor our data to decide whether its deployment ready and to check if there any issues with our model.

We will be mainly using Prometheus and Grafana for our monitoring while Prometheus can monitor our model Grafana can make dashboards so we can analyse it visually.

Prometheus:

```
global:
  scrape_interval: 15s

rule_files:
  - "alert_rules.yml"

scrape_configs:
  - job_name: "fastapi"
    static_configs:
      - targets: ["fastapi:8000"]
```

We have put scrape interval as 15s which means every 15 seconds Prometheus will access the fastapi and collect the metrics. We have given an alert rule here,

Alert rule:

```
groups:
  - name: leakage-alerts
    interval: 30s
    rules:
```

```
  - alert: DataLeakageWarning
    expr: data_leakage_gap > 0.2
    for: 2m
    labels:
      severity: warning
      service: fastapi
      environment: production
    annotations:
      summary: "Possible data leakage detected"
      description: "Train-test performance gap exceeded 0.2 for 2 minutes."

  - alert: DataLeakageCritical
    expr: data_leakage_gap > 0.5
    for: 1m
    labels:
      severity: critical
      service: fastapi
      environment: production
    annotations:
      summary: "Critical data leakage detected"
      description: "Train-test performance gap exceeded 0.5 for 1 minute."
```

groups is used to organise the alerts into logical sets. The name of the system is leakage-alerts.

We have created two sets here,

1. Data leakage warning:
   In here if the data leakage gap we explained in fastapi is greater than 0.2(20%) for 2 minutes it will show us a warning of Possible data leakage detected
2. Data leakage critical: in here if the data leakage gap has expanded more than 0.5 for 1 min it will show a severity warning of critical data leakage detected.

CI/CD(Github actions):

name: CI/CD Pipeline

```
on:
  push:
    branches:
          -main
```

Runs automatically whenever code is pushed to main branch

BUILD:

```
jobs:
  build:
    runs-on: ubuntu-latest
```

the build runs on ubuntu

```
    steps:
       uses: actions/checkout@v4
```

pulls our repository in to the runner so that it can access all of the files

```
       uses: docker/setup-buildx-action@v3
```

enable advanced docker build features

```
       uses: docker/login-action@v3
       with:
         registry: ghcr.io
         username: althu123696
         password: ${{ secrets.GHCR_TOKEN }}
```

The purpose of this step is to authenticate the GitHub Actions runner with GitHub's Container Registry (GHCR) so it can push Docker images.

The login username is althu123696 and password is secrets.GHCR_TOKEN token which is generated temporarily and it will be automatically given by github. It expires after the workflow ends.

```
     - name: Build and push FastAPI image
       run: |
         docker buildx build --push \
```

```
-t ghcr.io/althu123696/ipl-ii/fastapi:latest \
-f app/Dockerfile .
```

Build and push fastapi image using buildx, since we are using –push the image wont be saved locally after creation it will directly sent to ghcr.

```
    - name: Build and push Streamlit image
      run: |
        docker buildx build --push \
          -t ghcr.io/althu123696/ipl-ii/streamlit:latest \
          -f streamlit/Dockerfile .
```

It also have the same procedure as fastapi.

How it works:

Runs in to the project directory

Fetches the latest code from main branch

Pulls the newest docker image of streamlit and fastapi from ghcr

Restart the container in detached mode, applying all the updates.

STEPS AUTOMATED:

- Automatically pulls your repository into the GitHub Actions runner.
- Prepares the runner for advanced Docker builds.
- Log in to GitHub Container Registry (GHCR)
- Builds the backend Docker image from app/Dockerfile and pushes it to GHCR.
- Builds the frontend Docker image from streamlit/Dockerfile and pushes it to GHCR.

DOCKERIZATION:

Firstly we will create a dockerfile for fastapi and streamlit after that we can create a docker-compose file for adding Prometheus and Grafana into it since the image of Prometheus and Grafana is already on the docker hub

Fastapi:

```
FROM python:3.12-slim
WORKDIR /app
```

```
COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app/ /app

COPY model/ /app/model

EXPOSE 8000

CMD ["uvicorn", "api:app", "--host", "0.0.0.0", "--port", "8000"]
```

We are using python 3.12 version so from that we are telling the docker to set app as the working directory. After that we are installing the requirements that we installed for this project. Next we are copying fastapi codes from app inside the container. next we are copying our ml models files. The container will listen on port 8000. Docker will connect the container on the port 8000.

Streamlit:

```
FROM python:3.12-slim

WORKDIR /streamlit

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY streamlit/app.py .

EXPOSE 8501

CMD ["streamlit", "run", "app.py", "--server.port=8501", "--server.address=0.0.0.0"]
```

The same as fastapi docker but in here we are putting streamlit as working directory. Since the folder is different we will copy streamlit codes from streamlit folder. And the container will be connected to port 8501.

DOCKER-COMPOSE:

Dockerfile is used for creating a single image it only focuses on one single service while docker-compose is used for running and managing multiple containers.

```
services:
  fastapi:
    build:
      context: .
      dockerfile: app/Dockerfile
```

```
    image: ghcr.io/althu123696/ipl-ii/fastapi:latest
    container_name: fastapi
    restart: always
    ports:
      - "8000:8000"
    environment:
      - DEBUG=${DEBUG}
    volumes:
      - ./app/model:/app/model
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/"]
      interval: 30s
      timeout: 10s
      retries: 3
    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: "1g"

  streamlit:
    build:
      context: .
      dockerfile: streamlit/Dockerfile   # adjust path if needed
    image: ghcr.io/althu123696/ipl-ii/streamlit:latest
    container_name: streamlit
    restart: always
    ports:
      - "8501:8501"
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8501/"]
```

```
    interval: 30s
    timeout: 10s
    retries: 3

  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    restart: always
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - ./alert_rules.yml:/etc/prometheus/alert_rules.yml:ro
      - prometheus-data:/prometheus
    command: --config.file=/etc/prometheus/prometheus.yml
    depends_on:
      - fastapi
    healthcheck:
      test: ["CMD", "wget", "--spider", "http://localhost:9090/-/healthy"]
      interval: 30s
      timeout: 10s
      retries: 3

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    restart: always
    ports:
      - "3000:3000"
    volumes:
      - grafana-data:/var/lib/grafana
```

```
    - ./provisioning:/etc/grafana/provisioning
  environment:
    - GF_SECURITY_ADMIN_USER=${GF_SECURITY_ADMIN_USER}
    - GF_SECURITY_ADMIN_PASSWORD=${GF_SECURITY_ADMIN_PASSWORD}
  depends_on:
    - prometheus
  healthcheck:
    test: ["CMD", "wget", "--spider", "http://localhost:3000/api/health"]
    interval: 30s
    timeout: 10s
    retries: 3

mlflow:
  image: ghcr.io/mlflow/mlflow:latest
  container_name: mlflow
  restart: always
  ports:
    - "5000:5000"
  volumes:
    - ./mlruns:/mlflow/mlruns
    - ./mlflow:/mlflow
  command: >
    mlflow server
    --backend-store-uri file:/mlflow/mlruns
    --default-artifact-root /mlflow/mlruns
    --host 0.0.0.0
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:5000/"]
    interval: 30s
    timeout: 10s
    retries: 3
```

volumes:

prometheus-data:

grafana-data:

mlflow-data:

fast api:

Build the fast api image from the app/Dockerfile. Connect the container at port 8000. We mounted the model folder inside the container so that every time we update our local model the model in the container also update. This way we can use updated versions of the model every time.

Mlflow:

For mlflow we are going to use pre-built docker image from ghcr. It will take the latest version of mlflow packages into the container. We can access the mlflow ui at port 5000. Then we mounted the local folder mlruns into the container.

Mlflow allows cross origin so that other services like fastapi or streamlit can access it

Set the hostname of mlflow as mlflow.

Prometheus:

We are taking the latest image of Prometheus and it will be accessible in port 9090.

Then we stored the Prometheus.yml and alert rules Into the container.

Then we created a file Prometheus to stored its time-series data.

Grafana:

We are taking the latest image of Grafana and it can be accessed in port 3000.

In volume we created a persistent volume to store the data. In environment we created a username and password. And Grafana is dependent on Prometheus.
we can use Grafana to build dashboard to trach different problems we will face while deployment.

We created environment for Grafana so it is easy to store the userid and password.

Streamlit:

we are using the dockerfile present in the streamlit folder for frontend. We have took the image from our github container registry and the app will be available on port : 8501.

Images / ghcr.io/althu123696/ipl-internship/streamlit:latest

ghcr.io/althu123696/ipl-interns... IN USE

eea8af97a134

CREATED 10 hours ago

SIZE 1.33 GB

Layers (16)

Vulnerabilitie

0	# debian.sh --arch 'amd64' out/ 'trixie' '...	87.43 MB
1	ENV PATH=/usr/local/bin:/usr/local/s...	0 B
2	ENV LANG=C.UTF-8	0 B
3	RUN /bin/sh -c set -eux; apt-get updat...	4.94 MB
4	ENV GPG_KEY=7169605F62C751356...	0 B
5	ENV PYTHON_VERSION=3.12.12	0 B
6	ENV PYTHON_SHA256=fb85a13414b...	0 B
7	RUN /bin/sh -c set -eux; savedAptMar...	41.34 MB
8	RUN /bin/sh -c set -eux; for src in idle...	16.38 KB

You can u

Enable back

Images / ghcr.io/althu123696/ipl-internship/fastapi:latest

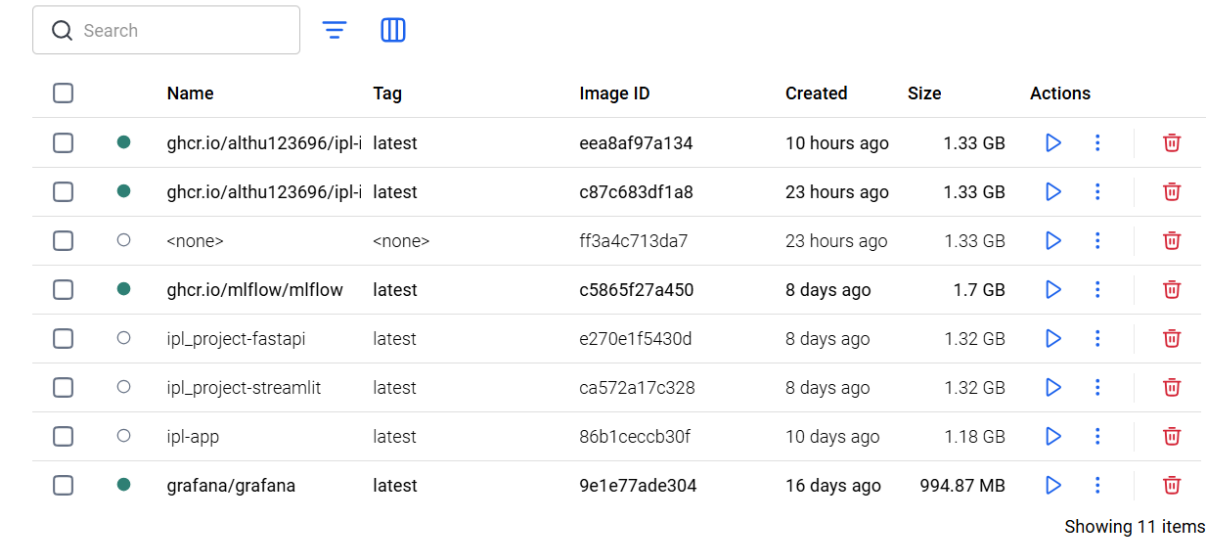
ghcr.io/althu123696/ipl-interns... IN USE

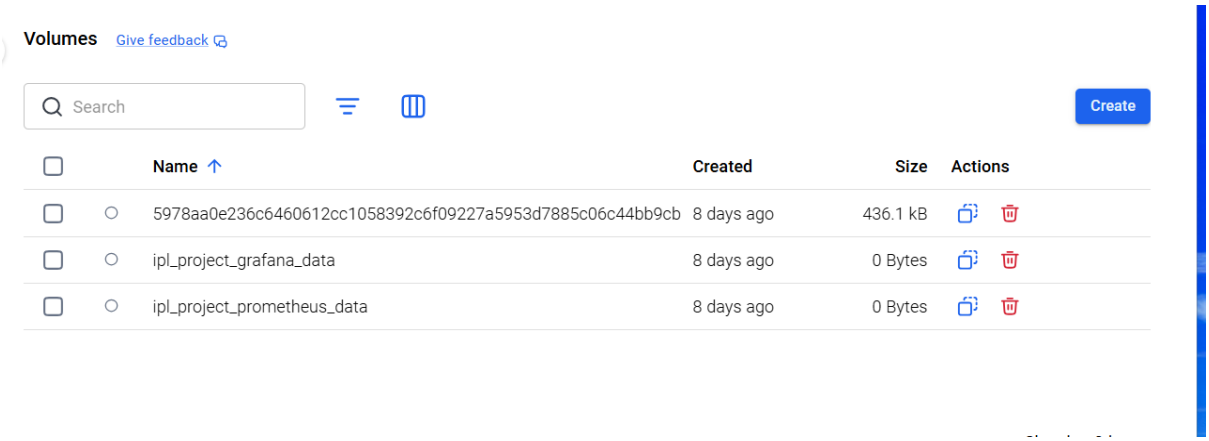
c87c683df1a8

CREATED 23 hours ago

Layers (17)

0	# debian.sh --arch 'amd64' out/ 'trixie' '...	87.43 MB
1	ENV PATH=/usr/local/bin:/usr/local/s...	0 B
2	ENV LANG=C.UTF-8	0 B
3	RUN /bin/sh -c set -eux; apt-get updat...	4.94 MB
4	ENV GPG_KEY=7169605F62C751356...	0 B
5	ENV PYTHON_VERSION=3.12.12	0 B
6	ENV PYTHON_SHA256=fb85a13414b...	0 B
7	RUN /bin/sh -c set -eux; savedAptMar...	41.34 MB
8	RUN /bin/sh -c set -eux; for src in idle...	16.38 KB


Search
Name
Tag
Image ID
Created
Size
Actions
ghcr.io/althu123696/ipl-i latest eea8af97a134 10 hours ago 1.33 GB
ghcr.io/althu123696/ipl-i latest c87c683df1a8 23 hours ago 1.33 GB
<none> <none> ff3a4c713da7 23 hours ago 1.33 GB
ghcr.io/mlflow/mlflow latest c5865f27a450 8 days ago 1.7 GB
ipl_project-fastapi latest e270e1f5430d 8 days ago 1.32 GB
ipl_project-streamlit latest ca572a17c328 8 days ago 1.32 GB
ipl-app latest 86b1ceccb30f 10 days ago 1.18 GB
grafana/grafana latest 9e1e77ade304 16 days ago 994.87 MB
Showing 11 items


Volumes
Give feedback
Search
Create
Name
Created
Size
Actions
5978aa0e236c6460612cc1058392c6f09227a5953d7885c06c44bb9cb 8 days ago 436.1 kB
ipl_project_grafana_data 8 days ago 0 Bytes
ipl_project_prometheus_data 8 days ago 0 Bytes


ipl_project
C:\Users\hp\Desktop\IPL_PROJECT
View configurations
streamlit
althu123696/ipl-internsl
8501:8501
fastapi
althu123696/ipl-internsl
8000:8000
grafana
grafana/grafana:latest
3000:3000
prometheus
prom/prometheus:lates
9090:9090
mlflow
mlflow/mlflow:latest
5000:5000
app=alerting-notifications
t=2026-02-28T05:34:04.798967142Z level=info caller=logger.go:214
time=2026-02-28T05:34:04.798951243Z msg="App initialized"
app=playlist
t=2026-02-28T05:34:04.801684286Z level=info caller=logger.go:214
time=2026-02-28T05:34:04.801668898Z msg="App initialized"
app=plugins
t=2026-02-28T05:34:04.803712695Z level=info caller=logger.go:214
time=2026-02-28T05:34:04.8036943Z msg="App initialized"
app=example
t=2026-02-28T05:34:04.805036091Z level=info caller=logger.go:214
time=2026-02-28T05:34:04.80501734Z msg="App initialized"
app=alerting-notifications
logger=backgroundsvcs.managerAdapter t=2026-02-28T05:34:04.805476646Z level=info msg=starting
module=plugins.store
logger=backgroundsvcs.managerAdapter t=2026-02-28T05:34:04.80586119Z level=info msg=starting
module=plugin.backgroundinstaller
logger=plugin.backgroundinstaller t=2026-02-28T05:34:04.806181227Z level=info msg="Plugins installed"
plugins=[]
logger=backgroundsvcs.managerAdapter t=2026-02-28T05:34:04.807004607Z level=info msg=starting
module=provisioning
logger=plugin.backgroundinstaller t=2026-02-
RAM 2.09 GB CPU 7.33% Disk: 27.32 GB used (limit 1006.85 GB)

To run the containers:

Docker-compose up -d: it will run the container in detached form.
individually we can command images with:

docker run -d -p <port:port> <full name of the image>

CONCLUSION AND FUTURE ENHANCEMENT:

The IPL player runs prediction project demonstrates the effective use of machine learning to forecast individual player performance. By analysing historical batting records, match contexts, and player-specific features, the system provides valuable insights into expected runs. The integration of FastAPI for serving predictions, Streamlit for visualization, and CI/CD pipelines for automation ensures a smooth developer experience and reliable deployment. This project highlights how data-driven approaches can enhance fan engagement and team decision-making.

Future enhancement:

- we can try automatic deployment using cd so that everytime we push into git the image will be build and using docker compose pull and up we can run the project container. So we don't have to start it manually.
- Add more data so the model will find the pattern accurately